Hotel Booking Platform Backend Assignment

Difficulty: (7/10) - no ai, feel free to use google

Time Limit: 2hr

Evaluation: Automated test cases will be run against your backend, link will be added below after exam for reference : https://github.com/rahul-MyGit/hotel-contest-test

---

Tech Stack (PREFERRED)

- Node.js, Express, PostgreSQL, JWT (Authentication), bcrypt (Password hashing), Zod

---

Objective

Build a **Hotel Booking Platform** backend where:

- **Hotel Owners** can list their properties and manage rooms
- **Customers** can search hotels, book rooms, and manage bookings
- All APIs follow **strict contracts** so automated tests can validate them

---

User Roles

Role	Description
`customer`	Books hotel rooms
`owner`	Manages hotels and rooms

Core Rules (Hard Rules)

1. One hotel can have multiple rooms with different types

2. Owners **cannot** book rooms in their own hotels
3. Bookings allowed **only for future dates**
4. No double booking - same room cannot be booked for overlapping dates
5. Customers can cancel bookings up to 24 hours before check-in
6. JWT required for **all APIs except signup/login**
7. Responses must match the format **exactly**

Database Schema (PostgreSQL)

- Below is not a strict schema that you need to follow, feel free to change it as per requirement or if you want to make application more normalised but make sure your response is matching the template.

users

```
CREATE TABLE users (
    id VARCHAR(255) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL CHECK (role IN ('customer', 'ow
ner')),
    phone VARCHAR(20),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

hotels

```
CREATE TABLE hotels (
    id VARCHAR(255) PRIMARY KEY,
    owner_id VARCHAR(255) REFERENCES users(id),
    name VARCHAR(255) NOT NULL,
    description TEXT,
```

```
    city VARCHAR(100) NOT NULL,
    country VARCHAR(100) NOT NULL,
    amenities JSONB DEFAULT '[]',
    rating DECIMAL(2,1) DEFAULT 0.0,
    total_reviews INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

rooms

```
CREATE TABLE rooms (
    id VARCHAR(255) PRIMARY KEY,
    hotel_id VARCHAR(255) REFERENCES hotels(id) ON DELETE CAS
CADE,
    room_number VARCHAR(50) NOT NULL,
    room_type VARCHAR(100) NOT NULL,
    price_per_night DECIMAL(10,2) NOT NULL,
    max_occupancy INTEGER NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(hotel_id, room_number)
);
```

bookings

```
CREATE TABLE bookings (
    id VARCHAR(255) PRIMARY KEY,
    user_id VARCHAR(255) REFERENCES users(id),
    room_id VARCHAR(255) REFERENCES rooms(id),
    hotel_id VARCHAR(255) REFERENCES hotels(id),
    check_in_date DATE NOT NULL,
    check_out_date DATE NOT NULL,
    guests INTEGER NOT NULL,
    total_price DECIMAL(10,2) NOT NULL,
    status VARCHAR(50) DEFAULT 'confirmed' CHECK (status IN
('confirmed', 'cancelled')),
```

```
    booking_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    cancelled_at TIMESTAMP,
    CONSTRAINT check_dates CHECK (check_out_date > check_in_d
ate)
);
```

reviews

```
CREATE TABLE reviews (
    id VARCHAR(255) PRIMARY KEY,
    user_id VARCHAR(255) REFERENCES users(id),
    hotel_id VARCHAR(255) REFERENCES hotels(id),
    booking_id VARCHAR(255) REFERENCES bookings(id),
    rating INTEGER NOT NULL CHECK (rating >= 1 AND rating <=
5),
    comment TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(user_id, booking_id)
);
```

Global API Rules

Auth Header
```
Authorization: Bearer <JWT_TOKEN>
```

Response Format (STRICT)

Success Response:

```
{
  "success": true,
```

```
  "data": {},
  "error": null
}
```

Error Response:

```
{
  "success": false,
  "data": null,
  "error": "ERROR_CODE"
}
```

- **No extra keys allowed**
- **No nested error objects**
- **Error must be a string code, not an object**

API Endpoints (10 Total)

1. POST /api/auth/signup

Register a new user (customer or owner)

Request Body

```
{
  "name": "Priya Sharma",
  "email": "priya@example.com",
  "password": "priya123",
  "role": "customer",
  "phone": "+919876543210"
}
```

Success Response – `201 Created`

```
{
  "success": true,
  "data": {
    "id": "usr_1a2b3c4d5e",
    "name": "Priya Sharma",
    "email": "priya@example.com",
    "role": "customer",
    "phone": "+919876543210"
  },
  "error": null
}
```

- If no role is given then default is `customer`
- Phone is optional

Error Responses

400 Bad Request – Invalid schema

```
{
  "success": false,
  "data": null,
  "error": "INVALID_REQUEST"
}
```

400 Bad Request – Email exists

```
{
  "success": false,
  "data": null,
  "error": "EMAIL_ALREADY_EXISTS"
}
```

---

2. POST /api/auth/login

Login and receive JWT token

Request Body

```
{
  "email": "priya@example.com",
  "password": "priya123"
}
```

Success Response – `200 OK`

```
{
  "success": true,
  "data": {
    "token": "JWT_TOKEN",
    "user": {
      "id": "usr_1a2b3c4d5e",
      "name": "Priya Sharma",
      "email": "priya@example.com",
      "role": "customer"
    }
  },
  "error": null
}
```

Error Responses

400 Bad Request – Invalid schema

```
{
  "success": false,
  "data": null,
  "error": "INVALID_REQUEST"
}
```

401 Unauthorized – Wrong credentials

```
{
  "success": false,
  "data": null,
  "error": "INVALID_CREDENTIALS"
}
```

---

3. POST /api/hotels – *(Owner Only)*

Create a new hotel

Headers: `Authorization: Bearer <token>`

Request Body

```
{
  "name": "Grand Palace Hotel",
  "description": "Luxury 5-star hotel in the heart of the cit
y",
  "city": "Mumbai",
  "country": "India",
  "amenities": ["wifi", "pool", "gym", "parking", "restauran
t"]
}
```

Success Response – `201 Created`

```
{
  "success": true,
  "data": {
    "id": "hotel_abc123",
    "ownerId": "usr_1a2b3c4d5e",
    "name": "Grand Palace Hotel",
    "description": "Luxury 5-star hotel in the heart of the c
```

```
ity",
    "city": "Mumbai",
    "country": "India",
    "amenities": ["wifi", "pool", "gym", "parking", "restaura
nt"],
    "rating": 0.0,
    "totalReviews": 0
  },
  "error": null
}
```

Error Responses

401 Unauthorized

```
{
  "success": false,
  "data": null,
  "error": "UNAUTHORIZED"
}
```

403 Forbidden – Not an owner

```
{
  "success": false,
  "data": null,
  "error": "FORBIDDEN"
}
```

400 Bad Request – Invalid schema

```
{
  "success": false,
  "data": null,
```

```
  "error": "INVALID_REQUEST"
}
```

---

4. POST /api/hotels/:hotelId/rooms – *(Owner Only)*

Add a room to a hotel

Headers: `Authorization: Bearer <token>`

Request Body

```
{
  "roomNumber": "101",
  "roomType": "Deluxe",
  "pricePerNight": 5000,
  "maxOccupancy": 2
}
```

Success Response – `201 Created`

```
{
  "success": true,
  "data": {
    "id": "room_xyz789",
    "hotelId": "hotel_abc123",
    "roomNumber": "101",
    "roomType": "Deluxe",
    "pricePerNight": 5000,
    "maxOccupancy": 2,
  },
  "error": null
}
```

Error Responses

401 Unauthorized

```
{
  "success": false,
  "data": null,
  "error": "UNAUTHORIZED"
}
```

403 Forbidden – Not the owner of this hotel

```
{
  "success": false,
  "data": null,
  "error": "FORBIDDEN"
}
```

400 Bad Request – Room number already exists

```
{
  "success": false,
  "data": null,
  "error": "ROOM_ALREADY_EXISTS"
}
```

400 Bad Request – Invalid schema

```
{
  "success": false,
  "data": null,
  "error": "INVALID_REQUEST"
}
```

404 Not Found – Hotel not found

```
{
  "success": false,
```

```
  "data": null,
  "error": "HOTEL_NOT_FOUND"
}
```

---

5. GET /api/hotels

Search and filter hotels (simple without pagination)

Headers: `Authorization: Bearer <token>`

Query Parameters (all optional if none provided return all hotels):

- `city` – Filter by city (case-insensitive)
- `country` – Filter by country (case-insensitive)
- `minPrice` – Minimum price per night
- `maxPrice` – Maximum price per night
- `minRating` – Minimum rating (0-5)

Example:

- `/api/hotels?city=Mumbai&minRating=4`
- `/api/hotels?country=India&minPrice=3000&maxPrice=8000`

Success Response – `200 OK`

```
{
  "success": true,
  "data": [
    {
      "id": "hotel_abc123",
      "name": "Grand Palace Hotel",
      "description": "Luxury 5-star hotel in the heart of the
city",
      "city": "Mumbai",
      "country": "India",
      "amenities": ["wifi", "pool", "gym"],
```

```
      "rating": 4.5,
      "totalReviews": 42,
      "minPricePerNight": 5000
    },
    {
      "id": "hotel_def456",
      "name": "Sea View Resort",
      "description": "Beautiful beach resort",
      "city": "Mumbai",
      "country": "India",
      "amenities": ["wifi", "pool", "beach"],
      "rating": 4.2,
      "totalReviews": 28,
      "minPricePerNight": 7000
    }
  ],
  "error": null
}
```

Field Calculations:

- `minPricePerNight` = minimum price_per_night among all rooms in this hotel.
- Exclude hotel with no rooms.
- `rating` and `totalReviews` come directly from the hotels table.

Error Responses

401 Unauthorized

```
{
  "success": false,
  "data": null,
  "error": "UNAUTHORIZED"
}
```

---

6. GET /api/hotels/:hotelId

Get detailed hotel information with all rooms

Headers: `Authorization: Bearer <token>`

Success Response – `200 OK`

```
{
  "success": true,
  "data": {
    "id": "hotel_abc123",
    "ownerId": "usr_owner123",
    "name": "Grand Palace Hotel",
    "description": "Luxury 5-star hotel in the heart of the c
ity",
    "city": "Mumbai",
    "country": "India",
    "amenities": ["wifi", "pool", "gym", "parking"],
    "rating": 4.5,
    "totalReviews": 42,
    "rooms": [
      {
        "id": "room_xyz789",
        "roomNumber": "101",
        "roomType": "Deluxe",
        "pricePerNight": 5000,
        "maxOccupancy": 2,
      },
      {
        "id": "room_xyz790",
        "roomNumber": "201",
        "roomType": "Suite",
        "pricePerNight": 10000,
        "maxOccupancy": 4,
      }
    ]
```

```
  },
  "error": null
}
```

- ownerId is visible to all users

Error Responses

401 Unauthorized

```
{
  "success": false,
  "data": null,
  "error": "UNAUTHORIZED"
}
```

404 Not Found

```
{
  "success": false,
  "data": null,
  "error": "HOTEL_NOT_FOUND"
}
```

---

7. POST /api/bookings – *(Customer Only)*

Create a new booking (Booking creation should be done atomically to prevent race conditions)

Headers: `Authorization: Bearer <token>`

Request Body

```
{
  "roomId": "room_xyz789",
  "checkInDate": "2026-02-15",
  "checkOutDate": "2026-02-18",
```

```
  "guests": 2
}
```

Success Response – `201 Created`

```
{
  "success": true,
  "data": {
    "id": "booking_123abc",
    "userId": "usr_1a2b3c4d5e",
    "roomId": "room_xyz789",
    "hotelId": "hotel_abc123",
    "checkInDate": "2026-02-15",
    "checkOutDate": "2026-02-18",
    "guests": 2,
    "totalPrice": 15000,
    "status": "confirmed",
    "bookingDate": "2026-01-22T10:30:00Z"
  },
  "error": null
}
```

Total Price Calculation:

```
nights = (new Date(checkOutDate) - new Date(checkInDate)) /
(1000 * 60 * 60 * 24)
totalPrice = nights * pricePerNight
```

Error Responses

401 Unauthorized

```
{
  "success": false,
  "data": null,
```

```
  "error": "UNAUTHORIZED"
}
```

403 Forbidden – Owner trying to book their own hotel

```
{
  "success": false,
  "data": null,
  "error": "FORBIDDEN"
}
```

400 Bad Request – Room not available (overlapping booking exists)

```
{
  "success": false,
  "data": null,
  "error": "ROOM_NOT_AVAILABLE"
}
```

400 Bad Request – Past date booking

```
{
  "success": false,
  "data": null,
  "error": "INVALID_DATES"
}
```

400 Bad Request – Guests capacity

```
{
  "success": false,
  "data": null,
  "error": "INVALID_CAPACITY"
}
```

400 Bad Request – Invalid schema

```
{
  "success": false,
  "data": null,
  "error": "INVALID_REQUEST"
}
```

404 Not Found

```
{
  "success": false,
  "data": null,
  "error": "ROOM_NOT_FOUND"
}
```

---

8. GET /api/bookings - (Customer only)

Get all bookings for current user

Headers: `Authorization: Bearer <token>`

Query Parameters (optional):

- `status` – Filter by status (confirmed/cancelled)

Success Response – `200 OK`

json

```
{
  "success": true,
  "data": [
    {
      "id": "booking_123abc",
      "roomId": "room_xyz789",
      "hotelId": "hotel_abc123",
      "hotelName": "Grand Palace Hotel",
      "roomNumber": "101",
```

```
      "roomType": "Deluxe",
      "checkInDate": "2026-02-15",
      "checkOutDate": "2026-02-18",
      "guests": 2,
      "totalPrice": 15000,
      "status": "confirmed",
      "bookingDate": "2026-01-22T10:30:00Z"
    }
  ],
  "error": null
}
```

Error Responses

401 Unauthorized

```
{
  "success": false,
  "data": null,
  "error": "UNAUTHORIZED"
}
```

---

9. PUT /api/bookings/:bookingId/cancel – *(Customer Only)*

Cancel a booking

Headers: `Authorization: Bearer <token>`

Success Response – `200 OK`

```
{
  "success": true,
  "data": {
```

```
    "id": "booking_123abc",
    "status": "cancelled",
    "cancelledAt": "2026-01-23T14:20:00Z"
  },
  "error": null
}
```

Error Responses

401 Unauthorized

```
{
  "success": false,
  "data": null,
  "error": "UNAUTHORIZED"
}
```

403 Forbidden – Not your booking

```
{
  "success": false,
  "data": null,
  "error": "FORBIDDEN"
}
```

400 Bad Request – Already cancelled

```
{
  "success": false,
  "data": null,
  "error": "ALREADY_CANCELLED"
}
```

400 Bad Request – Less than 24 hours before check-in

```
{
  "success": false,
  "data": null,
  "error": "CANCELLATION_DEADLINE_PASSED"
}
```

404 Not Found

```
{
  "success": false,
  "data": null,
  "error": "BOOKING_NOT_FOUND"
}
```

---

10. POST /api/reviews – *(Customer Only)*

Submit a review for a hotel after completing a booking

Headers: `Authorization: Bearer <token>`

Request Body

```
{
  "bookingId": "booking_123abc",
  "rating": 5,
  "comment": "Excellent service and beautiful rooms!"
}
```

Success Response – `201 Created`

```
{
  "success": true,
  "data": {
    "id": "review_xyz123",
    "userId": "usr_1a2b3c4d5e",
```

```
    "hotelId": "hotel_abc123",
    "bookingId": "booking_123abc",
    "rating": 5,
    "comment": "Excellent service and beautiful rooms!",
    "createdAt": "2026-02-19T10:00:00Z"
  },
  "error": null
}
```

NOTE: Can only review after check-out date has passed and booking status is confirmed
NOTE: Don't forget to update rating and totalRatings on success :-)
newRating = ((oldRating * totalReviews) + newRating) / (totalReviews + 1)

Error Responses

401 Unauthorized

```
{
  "success": false,
  "data": null,
  "error": "UNAUTHORIZED"
}
```

403 Forbidden – Not your booking

```
{
  "success": false,
  "data": null,
  "error": "FORBIDDEN"
}
```

400 Bad Request – Already reviewed

```
{
  "success": false,
```

```
  "data": null,
  "error": "ALREADY_REVIEWED"
}
```

400 Bad Request – Booking not eligible for review (check-out date not passed or cancelled)

```
{
  "success": false,
  "data": null,
  "error": "BOOKING_NOT_ELIGIBLE"
}
```

400 Bad Request – Invalid schema

```
{
  "success": false,
  "data": null,
  "error": "INVALID_REQUEST"
}
```

404 Not Found

```
{
  "success": false,
  "data": null,
  "error": "BOOKING_NOT_FOUND"
}
```

Implementation Guidelines

ID Generation

Use any method to generate unique string IDs:

- UUID v4
- nanoid
- Custom prefix + random string (e.g., `usr_`, `hotel_`, `room_`, `booking_`, `review_`)

DATE and TIME Validation Rules

IMORTANT: All date/time comparisons should use JavaScript's `new Date()` directly. No timezone conversions needed.

1. Check-in Date Must Be in Future

```
const today = new Date();
today.setHours(0, 0, 0, 0); // Reset to start of day

const checkIn = new Date(checkInDate); // e.g., "2026-02-15"

if (checkIn <= today) {
  // Return error: INVALID_DATES
}
```

Example:

- Today is January 22, 2026
- Check-in date "2026-01-22" → INVALID (not in future)
- Check-in date "2026-01-23" → VALID

---

2. Check-out Must Be After Check-in

```
const checkIn = new Date(checkInDate);
const checkOut = new Date(checkOutDate);

if (checkOut <= checkIn) {
  // Return error: INVALID_REQUEST
}
```

3. Cancellation Deadline (24 hours before check-in)

```
const now = new Date();
const checkIn = new Date(checkInDate);

const hoursUntilCheckIn = (checkIn - now) / (1000 * 60 * 60);

if (hoursUntilCheckIn < 24) {
  // Return error: CANCELLATION_DEADLINE_PASSED
}
```

Example:

- Current time: January 22, 2026 at 10:00 AM
- Check-in: January 23, 2026
- Hours until check-in: ~14 hours → CANNOT cancel
- Check-in: January 24, 2026
- Hours until check-in: ~38 hours → CAN cancel

4. Review Eligibility (after checkout)

A booking is eligible for review if:

1. The check-out date has passed
2. The booking status is 'confirmed' (not cancelled)

```
const today = new Date();
today.setHours(0, 0, 0, 0);

const checkOut = new Date(checkOutDate);

const canReview = checkOut < today && booking.status === 'con
firmed';
```

```
if (!canReview) {
  // Return error: BOOKING_NOT_ELIGIBLE
}
```

Example:

- Today: February 19, 2026
- Check-out: February 18, 2026, Status: confirmed → CAN review
- Check-out: February 20, 2026, Status: confirmed → CANNOT review (not past yet)
- Check-out: February 18, 2026, Status: cancelled → CANNOT review (cancelled)

Booking Conflict Detection

A room cannot be double-booked. Two bookings conflict if they share overlapping dates.

Overlap Condition:

```
// Existing booking in database
const existingCheckIn = new Date(existingBooking.check_in_dat
e);
const existingCheckOut = new Date(existingBooking.check_out_d
ate);

// New booking request
const newCheckIn = new Date(requestCheckIn);
const newCheckOut = new Date(requestCheckOut);

// Check for overlap
const hasOverlap = (newCheckIn < existingCheckOut) && (newChe
ckOut > existingCheckIn);

if (hasOverlap && existingBooking.status === 'confirmed') {
```

```
  // Return error: ROOM_NOT_AVAILABLE
}
```

Important: Only check for bookings with `status = 'confirmed'`. Ignore cancelled bookings.

Example Scenarios:

Existing Booking	New Booking	Overlap?
Jan 15 - Jan 18	Jan 18 - Jan 20	NO (checkout = check-in)
Jan 15 - Jan 18	Jan 16 - Jan 17	YES (fully inside)
Jan 15 - Jan 18	Jan 14 - Jan 16	YES (overlaps start)
Jan 15 - Jan 18	Jan 17 - Jan 20	YES (overlaps end)
Jan 15 - Jan 18	Jan 10 - Jan 14	NO (before)

Good luck!